Ustoz

Учёт оплат учебного центра / Тестовое задание Ecomdev / 29-10-2026

Рабочий прототип объединяет веб-панель, API, Cloudflare D1 и Telegram-бота. Администратор ведёт учеников, вносит оплаты и видит задолженность за выбранный месяц. Стек: TypeScript, React/Vite, Hono, Cloudflare Workers и grammY.

Ссылки для проверки

GitHub: https://github.com/shuhratartweb-blip/ecomdev-payments

Панель: https://ustoz.rieltor.store

Telegram: https://t.me/ustoz_ecomdev_2026_bot

Пароль панели передаётся проверяющим отдельно.

Бот доступен только разрешённым числовым Telegram ID в личном чате. Проверяющему необходимо заранее предоставить доступ через TELEGRAM_ALLOWED_IDS; одна ссылка не даёт права выполнять команды.

Соответствие заданию

Требование	Реализация
Часть 1. Ученики	Создание, список, изменение, удаление; имя, телефон, группа, месячный тариф.
Часть 1. Оплаты	Добавление суммы и даты, история по ученику, хранение в D1.
Часть 1. Должники	GET /debtors?month=YYYY-MM: ученики с положительным долгом и сумма задолженности.
Часть 2. Интерфейс	Выполнены оба варианта: адаптивный сайт и Telegram /debt, /pay.
Часть 3. README	Команды установки и запуска, проверки, 5 направлений улучшений.
Часть 4. ИИ	7 предложений о средствах, подсказках, ошибках и авторстве - в конце документа.

Время по оценке автора: 60 минут. Версия кода для проверок: 595c999. Платежи Payme/Click являются симуляцией без реальных списаний.

Веб-приложение

Интерфейс / Два языка / Две темы

Ученики

Карточка содержит ID, имя, телефон, группу и месячную сумму. Доступны создание, редактирование, удаление с подтверждением, поиск, фильтр группы и история оплат. Удаление ученика каскадно удаляет его платежи; архивирование пока не реализовано.

Должники и показатели

Администратор выбирает месяц. Карточки показывают общую сумму тарифов, поступившие оплаты и остаток задолженности. Таблица показывает тариф, оплаченную сумму и долг каждого должника. Из строки можно сразу открыть форму платежа.

Оплаты и история

Общий реестр показывает последние 500 записей. Для отдельного ученика доступна полная история. Учётная дата определяет месяц платежа, а время создания показывает момент внесения записи. Эти значения отображаются отдельно.

Ручная запись и симуляция

Для внесения оплаты выбираются ученик, сумма, дата и способ. Ручная запись фиксирует уже полученные деньги. Payme и Click открывают демонстрационное подтверждение с явной пометкой, что деньги не списываются. Успешное подтверждение сохраняет оплату и провайдера; отмена ничего не записывает.

Языки, темы и мобильный экран

Русский и узбекский латиницей переключаются в интерфейсе. Светлая тема использует синие акценты, тёплая - зелёные на кремовом фоне. Выбор сохраняется в браузере. На телефоне таблица учеников заменяется карточками, формы адаптированы к узкому экрану.

Демонстрационный режим

Параметр ?preview=1 включает отдельно помеченные вымышленные данные. Изменения действуют до перезагрузки страницы, не отправляются в рабочую базу и не становятся реальными платежами. Обычный режим использует API и требует входа.

Текущие правила: только целые UZS; даты в интерфейсе DD-MM-YYYY; время HH:mm:ss по Asia/Tashkent. API сохраняет даты в ISO-формате.

Telegram-бот

Команды задания и дополнительные кнопочные сценарии

Команда	Результат
/debt	Должники текущего месяца.
/pay 1 250000	Запись 250 000 UZS для ученика с ID 1.
/lang ru или /lang uz	Выбор языка, сохранённый в D1.
/help, /start	Справка и меню управления.

Все ученики и должники

Меню содержит отдельные разделы «Должники» и «Все ученики». Общий список показывает тариф, оплаченные суммы, долг либо переплату за текущий месяц. Можно выбрать ученика без долга и внести дополнительную оплату. Списки разбиты на страницы по 8 человек.

Внесение оплаты кнопками

1. Выбрать ученика. 2. Ввести положительную целую сумму. 3. Выбрать обычную запись либо Payme/Click. 4. Подтвердить. Бот сохраняет состояние незавершённого ввода. Возврат в меню отменяет операцию.

Payme и Click в Telegram

Для обоих способов есть отдельный экран симуляции и отдельная кнопка подтверждения. До подтверждения в базе нет записи. Устаревшая кнопка ручного подтверждения не должна принимать симуляцию. После сохранения чек содержит способ, отметку симуляции и дату/время. Повторная доставка подтверждения не создаёт дубликат.

Доступ и время

Требуются токен отдельного бота, список разрешённых ID и секрет webhook. Команды учёта разрешены только указанным пользователям в личном чате. Для команды /pay дата берётся из времени исходного сообщения в Asia/Tashkent, поэтому поздняя повторная доставка не меняет учётный день.

Самостоятельная настройка

Создать бота через @BotFather /newbot. Задать TELEGRAM_BOT_TOKEN, TELEGRAM_ALLOWED_IDS и TELEGRAM_WEBHOOK_SECRET в Worker. Для скрипта настройки экспортировать токен и секрет в окружение текущего терминала: .dev.vars он автоматически не читает.

```
APP_URL=https://YOUR_APP_DOMAIN node scripts/configure-bot.mjs
```

Без Telegram-настроек веб-панель и API продолжают работать. Существовавшие ранее боты не используются этим проектом.

API и расчёт долга

Часть 1 / Контракт / Точность учёта

Метод и путь	Назначение
POST /api/login; GET /api/session	Вход и проверка сессии. POST /api/logout - выход.
GET /api/students	Список учеников.
POST /api/students	Создание: name, phone, group, monthlyFee.
PATCH /api/students/:id	Изменение полей ученика.
DELETE /api/students/:id	Удаление ученика и связанных оплат.
GET, POST /api/students/:id/payments	История и внесение оплаты.
GET /api/payments	Последние 500 оплат.
GET /api/debtors?month=YYYY-MM	Только ученики с положительным долгом.
GET /api/balances?month=YYYY-MM	Балансы всех учеников для сводки.

Маршруты учёта также работают без /api, включая требуемый заданием GET /debtors. Пример тела платежа:

{"amount":"250000","date":"2026-10-29","method":"manual"}

Правила расчёта

Долг = max(текущий месячный тариф - сумма оплат выбранного месяца, 0). Учитываются даты от первого дня месяца включительно до первого дня следующего месяца исключительно. Ученик без оплат должен весь тариф; полностью оплативший и переплативший не включаются в должников.

Пример: тариф 700 000 UZS, платежи 250 000 и 150 000 UZS в октябре дают долг 300 000 UZS за октябрь. Платёж в сентябре на этот расчёт не влияет.

Валидация и повторные запросы

JSON-суммы - строки целых чисел; .00 допускается для совместимости. Ненулевые дроби отклоняются. Тариф может быть нулём, платёж должен быть положительным; максимум одного значения - 1 000 000 000 UZS. Проверяются календарные даты, месяц, типы полей и существование ученика.

Idempotency-Key сохраняется при повторе той же операции. Повтор возвращает существующий платёж; изменённое содержимое с тем же ключом даёт 409. Ошибки имеют форму {error:{code}}: 400 неверный ввод, 401 нет входа, 403 запрет, 404 ученик не найден, 409 конфликт, 429 лимит входа.

Архитектура и хранение

TypeScript / Cloudflare Workers / D1

React/Vite → Hono API в Cloudflare Worker → Cloudflare D1. Telegram webhook обращается к общей серверной логике учёта. Worker также раздаёт собранный интерфейс. Домен проекта - отдельный поддомен ustoz.rieltor.store.

Компонент	Файлы и назначение
Веб-интерфейс	src/web/App.tsx, forms.tsx, components.tsx, styles.css: страницы, формы, темы.
Клиент и локализация	client.ts, model.ts, i18n.ts, demo.ts: API, форматы, RU/UZ, демонстрация.
Сервер	src/server/index.ts, service.ts, domain.ts: маршруты, учёт и проверка данных.
Авторизация и бот	auth.ts, telegram.ts: сессии, команды и кнопочные сценарии.
Инфраструктура	wrangler.jsonc, migrations/, scripts/, .github/workflows/check.yml.

Таблицы базы

Таблица	Содержимое
students	Имя, телефон, группа, тариф, время создания.
payments	Ученик, сумма, дата, метод, уникальный ключ повтора, время создания.
telegram_preferences	Сохранённый язык пользователя бота.
telegram_flow	Состояние пошаговой оплаты.
login_attempts	Учёт попыток входа для ограничения перебора.

D1 использует SQLite. Суммы хранятся целыми минимальными единицами, хотя внешний ввод ограничен целыми UZS. Внешний ключ связывает платёж с учеником, индекс по ученику и дате ускоряет историю и месячный расчёт. Создание/изменение схемы выполняется SQL-миграциями.

Базовая защита

Сессии подписаны; HTTPS-cookie имеет HttpOnly, Secure, SameSite=Strict. Проверяется срок сессии и Origin изменяющих запросов, ограничиваются попытки входа. Telegram проверяет секрет webhook и список разрешённых пользователей. Секреты не должны попадать в Git. Общий пароль остаётся ограничением прототипа: персональных ролей пока нет.

Запуск с нуля

Нужны Git, Node.js 22+ (проверено на 24) и npm. Для локального сайта аккаунт Cloudflare не требуется.

```
git clone https://github.com/shuhratartweb-blip/ecomdev-payments.git
cd ecomdev-payments
npm ci
cp .env.example .dev.vars
node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"
```

В .dev.vars установить свой APP_PASSWORD и сгенерированный SESSION_SECRET (не менее 32 символов). Telegram-поля можно оставить пустыми. Файл исключён из Git.

```
npm run db:migrate
npm run db:seed   # необязательно, только новая пустая БД
npm run dev
```

Открыть http://localhost:8787 и войти с локальным APP_PASSWORD. База хранится в .wrangler/state/ и переживает перезапуск. Seed содержит вымышленные данные и платежи текущего месяца Ташкента.

```
# Во втором терминале: интерфейс с горячим обновлением
npm run web:dev
# http://localhost:5173, API проксируется на Worker 8787

npm run typecheck
npm test
npm run build
```

Развёртывание в своём Cloudflare

После создания D1 заменить database_id в wrangler.jsonc. Заменить или удалить routes для собственного домена. Выполнить удалённые миграции, задать секреты, затем публиковать.

```
npx wrangler login
npx wrangler d1 create ecomdev-payments
# Теперь заменить database_id и routes в wrangler.jsonc
npx wrangler d1 migrations apply DB --remote
npx wrangler secret put APP_PASSWORD
npx wrangler secret put SESSION_SECRET
# Для бота: TELEGRAM_BOT_TOKEN, TELEGRAM_WEBHOOK_SECRET,
# TELEGRAM_ALLOWED_IDS через wrangler secret put
npm run deploy
```

Cloudflare-ключ нужен инструменту публикации; в код приложения он не встраивается. Рабочий пароль не публикуется в открытом GitHub.

Проверки и ограничения

Результаты на 29-10-2026 / План ещё на неделю

Подтверждённые результаты

На версии 595c999 выполнены npm run typecheck, npm test и npm run build: 34 теста прошли, 0 ошибок; проверка типов и production-сборка успешны. Тесты используют изолированные локальные базы D1 через Miniflare. Отдельно проверен вход в опубликованную панель: HTTP 200. Рабочие оплаты при подготовке отчёта не создавались.

Покрыты CRUD учеников, расчёт долга, границы месяцев и лет, нулевой тариф, полная оплата и переплата, неверные данные, восемь конкурентных повторов одного платежа, сессии и лимит входа, Telegram-команды, кнопки, списки и симуляции. Проверены целые UZS, даты DD-MM-YYYY и время Ташкента.

Ранее в браузере проверены темы, RU/UZ, мобильные ширины 320/390 px, частичная оплата и отмена. Это исторические проверки интерфейса, а не новый полный прогон последней версии. Автотесты и локальная сборка не заменяют проверку всех сценариев на опубликованном сайте.

Что не реализовано и что ограничено

Реальных интеграций Payme/Click нет. Нет ролей, истории тарифов, начислений за неполный месяц, распределения платежей по периодам и переноса переплат. Удаление ученика удаляет оплаты. Общий реестр ограничен 500 последними записями. Проверяющему бота требуется разрешённый Telegram ID. Полный запуск README на чистой машине в этом проходе не повторялся.

Пять улучшений за дополнительную неделю

1. Персональные аккаунты, роли администратора/кассира/наблюдателя и управление доступом проверяющих.

2. История тарифов, периоды обучения и начисления с учётом даты начала занятий.

3. Распределение оплат по месяцам, перенос переплат и возвраты; подготовка настоящих Payme/Click-интеграций при наличии мерчант-доступа.

4. Аудит изменений, архивирование вместо удаления, экспорт отчётов.

5. Проверка восстановления резервных копий, мониторинг и дополнительные автоматические браузерные сценарии.

ИИ и передача задания

Часть 4 / Текст для Telegram / Комплект сдачи

Я использовал Codex для генерации API, React-интерфейса, Telegram-бота, миграций и тестов, а Image Gen — для вариантов визуального макета.

Лучше всего работали конкретные запросы с проверяемым результатом, например: «добавь два языка — узбекский латиницей и русский» и «добавь логотипы Payme и Click, а также симуляцию оплаты».

Работу разделили на веб-интерфейс и серверную часть, а результат проверяли сборкой, автоматическими тестами и действиями в браузере.

ИИ ошибся в отображении узбекского месяца: при браузерной проверке вместо названия появлялось M10, поэтому форматирование заменили явными названиями месяцев и добавили регрессионный тест.

Также тесты выявили, что JSON null мог приводить к серверной ошибке; после этого добавили проверку тела запроса, а повторные и конкурентные оплаты проверили на отсутствие дубликатов.

Моя роль состояла в постановке задачи, выборе вариантов интерфейса и уточнении поведения приложения; утверждать, что я самостоятельно написал какие-либо модули кода, было бы неверно.

Код и тесты в этой работе создавались и исправлялись с помощью ИИ, а реальные платёжные интеграции остались явно обозначенной симуляцией.

Что передать в Telegram @code_debugger007

Репозиторий: https://github.com/shuhratartweb-blip/ecomdev-payments

Панель: https://ustoz.rieltor.store

Бот: https://t.me/ustoz_ecomdev_2026_bot

Пароль панели передаётся проверяющим отдельно.

Время: 60 минут, по оценке автора. Текст об ИИ приведён выше (7 предложений). Все пункты частей 1-3 представлены в коде и README; оба варианта интерфейса реализованы. Ограничения и отсутствие реальных платёжных интеграций описаны на предыдущей странице.

Для проверки Telegram необходимо согласовать числовой ID и добавить его в список разрешённых пользователей. Сам документ и сообщение автоматически адресату не отправлялись.